

Task 1.1

```
#TASK 1.1

class Shape:
    def displayShape(self):
        print("Shape is displaying")

class Rectangle (Shape):
    def displayRectangle(self):
        print("Rectangle is displaying")

rect=Rectangle()
rect.displayShape()
rect.displayRectangle()
```

Output:

```
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\Assignments\Practical9\shape.py"
Shape is displaying
Rectangle is displaying
PS C:\TBC\Sem2\POP_Project>
```

Task 1.2

```
# TASK 1 (Modifying Class)

class Shape:
    #Constructor for Shape class
    def __init__(self):
        print("Shape constructor is called")
    #Method of Shape class
    def displayShape(self):
        print("Shape is displaying")

class Rectangle (Shape):
    #Constructor for Rectangle class
    def __init__(self):
        print("\nRectangle constructor is called")

    #Method of Rectangle class
    def displayRectangle(self):
        print("Rectangle is displaying")

shape = Shape()
shape.displayShape()
rect=Rectangle()
rect.displayShape()
rect.displayRectangle()
```

Output:

```
● PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\
Assignments\Practical9\tempCodeRunnerFile.py"
Shape constructor is called
Shape is displaying

Rectangle constructor is called
Shape is displaying
Rectangle is displaying
○ PS C:\TBC\Sem2\POP_Project>
```

Task 1.3:

```
#TASK 1 (Modifying Class)

class Shape:
    #Constructor for Shape class
    def __init__(self):
        print("Shape constructor is called")
    #Method of Shape class
    def displayShape(self):
        print("Shape is displaying")

class Rectangle (Shape):
    #Constructor for Rectangle class
    #Sub class constructor is called using 'super'
    def __init__(self):
        super().__init__()
        print("Rectangle constructor is called")

    #Method of Rectangle class
    def displayRectangle(self):
        print("Rectangle is displaying")

#shape = Shape()
#shape.displayShape()
rect=Rectangle()
#Super class method
rect.displayShape()
#Sub class method
rect.displayRectangle()

"""
The sub class (Rectangle class) is calling the super class(Shape Class)
constructor and method.
Therefore, while instantiating and creating object it is printing the super class
constructor and method along with it's own.
"""
```

Output:

```
Assignments\Practical9\shape.py"
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Project\
Assignments\Practical9\shape.py"
Shape constructor is called
Rectangle constructor is called
Shape is displaying
Rectangle is displaying
PS C:\TBC\Sem2\POP_Project>
```

Col 24 Spaces: 4 UTF-8 CRLF { } Python 

Task 1.4:

```
# TASK 1 (Modifying Class)

class Shape:
    #Constructor for Shape class
    def __init__(self,colour="Black"):
        self.colour =colour
        print("Shape constructor is called")

    #Method of Shape class
    def displayShape(self):
        print("Shape is displaying")

    def getColour(self):
        return self.colour

class Rectangle (Shape):
    #Constructor for Rectangle class
    #Sub class constructor is called using 'super'
    def __init__(self):
        super().__init__()
        print("Rectangle constructor is called")

    #Method of Rectangle class
    def displayRectangle(self):
        print("Rectangle is displaying")

    def getRectangleColour(self):
        #returns the base class color
        return self.colour

#shape = Shape()
#shape.displayShape()
rect=Rectangle()
#Super class method
#rect.displayShape()
#Sub class method
#rect.displayRectangle()
"""
The sub class (Rectangle class) is calling the super class(Shape Class)
constructor and method.
Therefore, while instantiating and creating object it is printing the super class
constructor and method along with it's own.
"""
```

```
print("Color:",rect.getRectangleColour())
```

Output:

```
ape.py"
Color: Black
● PS C:\TBC\Sem2\POP_Project> python
mpCodeRunnerFile.py"
Shape constructor is called
Rectangle constructor is called
Color: Black
○ PS C:\TBC\Sem2\POP_Project>
```

Task 1.5:

```
# TASK 1 (Modifying Class)

class Shape:
    #Constructor for Shape class
    def __init__(self,colour="Black"):
        self.__colour =colour
        print("Shape constructor is called")

    #Method of Shape class
    def displayShape(self):
        print("Shape is displaying")

    def getColour(self):
        return self.__colour

class Rectangle (Shape):
    #Constructor for Rectangle class
    #Sub class constructor is called using 'super'
    def __init__(self):
        super().__init__()
        print("Rectangle constructor is called")

    #Method of Rectangle class
    def displayRectangle(self):
        print("Rectangle is displaying")

    def getRectangleColour(self):
        #returns the base class color
        return self.colour

#shape = Shape()
#shape.displayShape()
rect=Rectangle()
#Super class method
#rect.displayShape()
#Sub class method
#rect.displayRectangle()
"""
Subclass does not inherit the private members of it's base class, but private
members can be accessed by base class public or protected methods.
Therefore, using getRectangleColour() (returning the base class method).
"""
print("Color:",rect.getRectangleColour())
```

Output:

```
TypeError: str object is not callable
● PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2
  nts\Practical9\shape.py"
  Shape constructor is called
  Rectangle constructor is called
  Color: Black
○ PS C:\TBC\Sem2\POP_Project>
```

main* 0 2 Ln 29, Col 1 Spaces: 4 UTF-8 CRLF

Task 1.6

```
# TASK 1 (Modifying Class)

class Shape:
    #Constructor for Shape class
    def __init__(self,colour="Black"):
        self._colour =colour
        print("Shape constructor is called")

    #Method of Shape class
    def displayShape(self):
        print("Shape is displaying")

    def getColour(self):
        return self._colour

class Rectangle (Shape):
    #Constructor for Rectangle class
    #Sub class constructor is called using 'super'
    def __init__(self):
        super().__init__()
        print("Rectangle constructor is called")

    #Method of Rectangle class
    # Overriding method of the base class
    def display(self):
        print("Rectangle is displaying")

    def getRectangleColour(self):
        #returns the base class color
        return self.colour

#shape = Shape()
#shape.display()
rect=Rectangle()

#Super class method
#rect.displayShape()
#Sub class method
#rect.displayRectangle()
"""
Protected members can be accessed by the sub class directly.
"""
#print(rect.colour)
```

```
print("Color:",rect.getRectangleColour())
```

Output:

```
nts\Practical9\shape.py"
Shape constructor is called
Rectangle constructor is called
Color: Black
PS C:\TBC\Sem2\POP_Project>
```

Task 1.7

```
# TASK 1 (Modifying Class)

class Shape:
    #Constructor for Shape class
    def __init__(self,colour="Black"):
        self.colour =colour
        print("Shape constructor is called")

    #Method of Shape class
    def display(self):
        print("Shape is displaying")

    def getColour(self):
        return self.colour

class Rectangle (Shape):
    #Constructor for Rectangle class
    #Sub class constructor is called using 'super'
    def __init__(self):
        super().__init__()
        print("Rectangle constructor is called")

    #Method of Rectangle class
    # Overriding method of the base class
    def display(self):
        print("Rectangle is displaying")

    def getRectangleColour(self):
        #returns the base class color
        return self.colour

#shape = Shape()
#shape.display()
rect=Rectangle()

#Super class method
#rect.displayShape()
#Sub class method
#rect.displayRectangle()
#print(rect.colour)
print("Color:",rect.getRectangleColour())
rect.display() #Overrides the base class method by the sub class
```

```
"""
```

Instead of displaying the shape class's method output, it is displaying the rectangle class method output because the sub class is overriding the base class's method.

```
"""
```

Output:

```
PS C:\TBC\Sem2\POP_Project> python -u C:\TBC\Sem2\POP_Project\Practical9\shape.py
Shape constructor is called
Rectangle constructor is called
Color: Black
Rectangle is displaying
PS C:\TBC\Sem2\POP_Project>
```

Task 1.8

```
# TASK 1 (Modifying Class)

class Shape:
    #Constructor for Shape class
    def __init__(self,colour="Black"):
        self.colour =colour
        print("Shape constructor is called")

    #Method of Shape class
    def display(self):
        print("Shape is displaying")

    def getColour(self):
        return self.colour

class Rectangle (Shape):
    #Constructor for Rectangle class
    #Sub class constructor is called using 'super'
    def __init__(self):
        super().__init__()
        print("Rectangle constructor is called")

    #Method of Rectangle class
    # Overriding method of the base class
    def display(self):
        print("Rectangle is displaying")

    def getRectangleColour(self):
        #returns the base class color
        return self.colour

class Square(Shape):
    pass

#shape = Shape()
#shape.display()
rect=Rectangle()

#print(rect.colour)
print("Color:",rect.getRectangleColour())
rect.display() #Overrides the base class method by the sub class.
print("\n")
sqr = Square()
```

```
sqr.display()
```

```
#the Square class doesnot overrides the method of base class which is why  
#while instantiating and creating object it display's base class method and  
constructor
```

Output:

```
● PS C:\TBC\Sem2\POP_Project> python -u "C:\TBC\Sem2\POP_Project\Practical9\shape2.py"  
Shape constructor is called  
Rectangle constructor is called  
Color: Black  
Rectangle is displaying  
  
Shape constructor is called  
Shape is displaying  
○ PS C:\TBC\Sem2\POP_Project> █
```

Task 1.9

```
# TASK 1 (Modifying Class)

class Shape:
    #Constructor for Shape class
    def __init__(self,colour="Black"):
        self.colour =colour
        print("Shape constructor is called")

    #Method of Shape class
    def display(self):
        print("Shape is displaying")

    def getColour(self):
        return self.colour

class Rectangle (Shape):
    #Constructor for Rectangle class
    #Sub class constructor is called using 'super'
    def __init__(self):
        super().__init__()
        print("Rectangle constructor is called")

    #Method of Rectangle class
    # Overriding method of the base class
    def display(self):
        print("Rectangle is displaying")

    def getRectangleColour(self):
        #returns the base class color
        return self.colour

class Square(Shape):
    pass

#shape = Shape()
#shape.display()
rect=Rectangle()

#print(rect.colour)
print("Color:",rect.getRectangleColour())
rect.display() #Overrides the base class method by the sub class.
print("\n")
sqr = Square()
```

```
sqr.display()
#the Square class doesnot overrides the method of base class which is why
#while instantiating and creating object it display's base class method and
constructor

sqr.getRectangleColour()
"""
sqr object is calling the Rectangle class method, but the Square class is the sub
class of Shape but the Rectangle class.
Therefore, it throws error while calling Rectangle class method by Squire class
object.
"""
```

Output:

```
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Project\Weekly Assignme
nts\Practical9\shape2.py"
Shape constructor is called
Rectangle constructor is called
Color: Black
Rectangle is displaying

Shape constructor is called
Shape is displaying
Traceback (most recent call last):
  File "c:\TBC\Sem2\POP_Project\Weekly Assignments\Practical9\shape2.py", line
  48, in <module>
    sqr.getRectangleColour()
    ^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'Square' object has no attribute 'getRectangleColour'
PS C:\TBC\Sem2\POP_Project> 
```

Task 2.1

```
#Task 2.1 Person.py
class Person:
    def __init__(self,firstName="Mr", lastName="X", address= "Brostol"):
        #Private variables
        self.__firstName= firstName
        self.__lastName= lastName
        self.__address= address

    #Setters
    def setFirstName(self,firstName):
        self.__firstName = firstName

    def setLastName(self, lastName):
        self.__lastName = lastName

    def setAddress(self,address):
        self.__address = address

    #Getters
    def getFirstName(self):
        return self.__firstName

    def getLastName(self):
        return self.__lastName

    def getAddress(self):
        return self.__address
    #returning string to display output
    def __str__(self):
        return "\n" + "Person's name is " + self.getFirstName() + " "
        "+self.getLastName()+"\nAddress is " +self.getAddress()

# Creating a person object with default values
print("Creating a person object with default values")
p=Person()

#Prints the default given arguments.
print(p.__str__())

#setting name and address
print("\nNow setting the names and address\n")
p.setFirstName("Abdur")
p.setLastName("Rakib")
```

```

p.setAddress("UWE Frenchay Campus 4QXX")

#Printing the information using __str__ method
print("Printing the information using __str__ method")
print(p)

#Creating another object by passing the values explicitly
print("\nCreating another Person object by passing the values explicitly")
p1=Person("Jun","Hong","UWE Frenchay Campus 3QXX")

#Printing the information using __str__ method
print("\nPrinting the information using __str__ method")
print(p1.__str__())

```

Output:

```

Address is UWE Frenchay Campus 3QXX
● PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Project\Weekly Assignments\Practical9\person.py"
Creating a person object with default values

Person's name is Mr X
Address is Brostol

Now setting the names and address

Printing the information using __str__ method

Person's name is Abdur Rakib
Address is UWE Frenchay Campus 4QXX

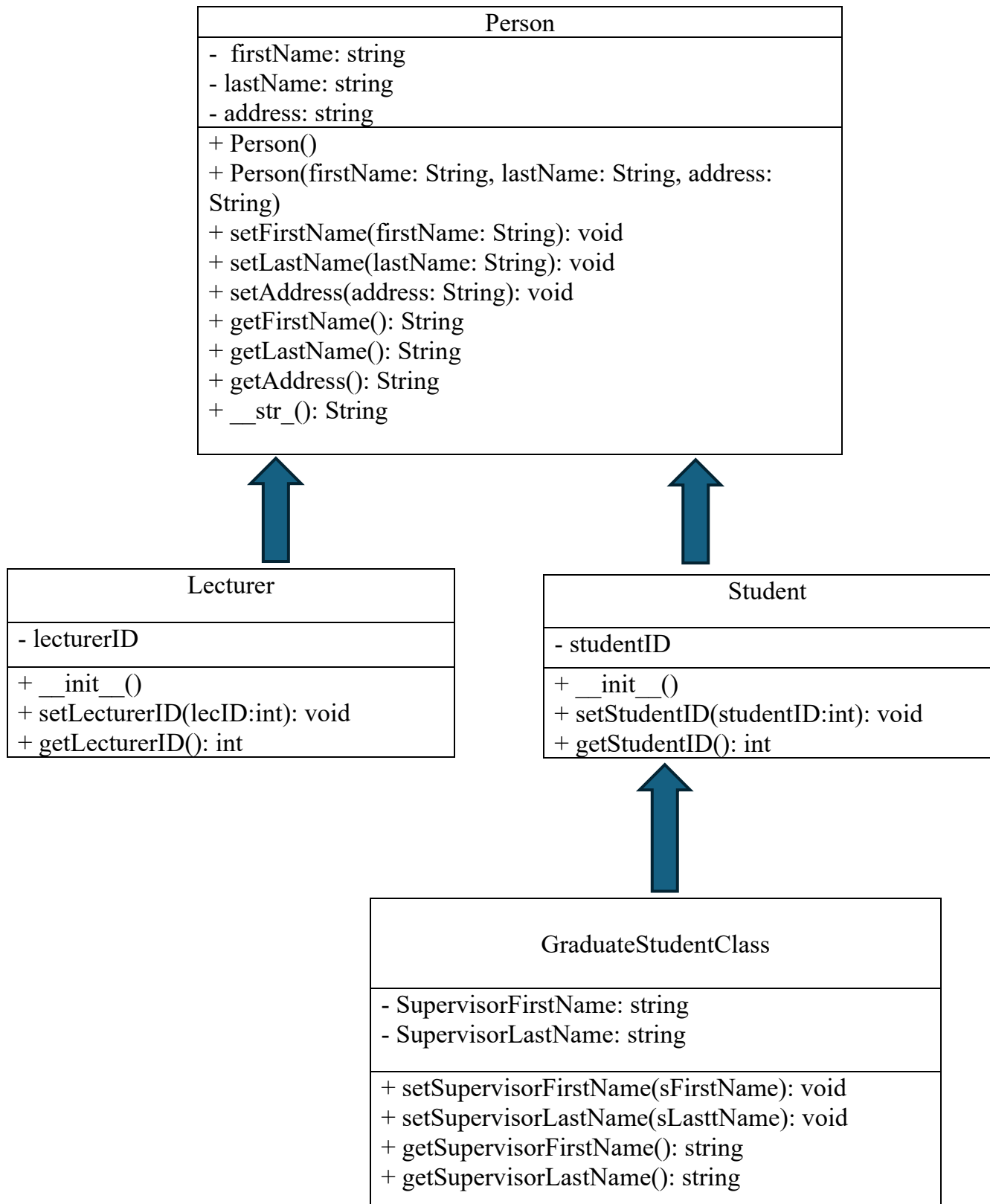
Creating another Person object by passing the values explicitly

Printing the information using __str__ method

Person's name is Jun Hong
Address is UWE Frenchay Campus 3QXX
○ PS C:\TBC\Sem2\POP_Project>

```

Task 2.2: UML Diagram



```

#Task 2.2 Person.py
class Person:
    def __init__(self,firstName="Mr/Ms", lastName="X", address= "Bristol"):
        #Private variables
        self.__firstName= firstName
        self.__lastName= lastName
        self.__address= address

    #Setters
    def setFirstName(self,firstName):
        self.__firstName = firstName

    def setLastName(self, lastName):
        self.__lastName = lastName

    def setAddress(self,address):
        self.__address = address

    #Getters
    def getFirstName(self):
        return self.__firstName

    def getLastName(self):
        return self.__lastName

    def getAddress(self):
        return self.__address
    #returning string to display output
    def __str__(self):
        return "\n" + "Person's name is " + self.getFirstName() + "
" + self.getLastName() + "\nAddress is " + self.getAddress()

class Lecturer(Person):

    def
__init__(self,lecID=None,lecFirstName="Mr/Ms",lecLastName="X",lecAddress="Bristol
"):
        super().__init__(lecFirstName,lecLastName,lecAddress)

        self.__lecturerID=lecID

    #Setters
    def setLecturerID(self,lecID):

```

```

        self.__lecturerID=lecID

#Getters
def getLecturerID(self):
    return self.__lecturerID

#Base class already have name and address getter setter and Lecturer class
has inherited it so no need to of new getter setter that's already in there

#Overriding Base class method
def __str__(self):
    #Using base method
    return (f"\nLecturer Name is {self.getFirstName()}
{self.getLastName()}\n"
            f"Address is {self.getAddress()}\n"
            #Sub class method
            f"Lecturer ID: {self.getLecturerID()}")

#Person parent class
class Student(Person):

    def
__init__(self,studentFirstName="Mr/Ms",studentLastName="X",studentAddress="Bristo
l",studentID=None):
    # Calling parent class constructor
    super().__init__(studentFirstName,studentLastName,studentAddress)

    self.__studentID=studentID

#Setters
def setStudentID(self,lecID):
    self.__studentID=lecID

#Getters
def getStudentID(self):
    return self.__studentID

#Base class already have name and address getter setter and student class has
inherited it so no need to of new getter setter that's already in there

#Overriding Base class method
def __str__(self):
    #Using base method
    return (f"\nStudent Name is {self.getFirstName()} {self.getLastName()}\n"
            f"Address is {self.getAddress()}\n")

```

```

        #Sub class method
        f"Student ID: {self.getStudentID()}")

#Using Student class as it's base
class GraduateStudentClass(Student):
    def
__init__(self,studentFirstName="Mr/Ms",studentLastName="X",studentAddress="Bristo
l",studentID=None,sFirstName="Mr/Ms",sLastName="X"):
    super().__init__(studentFirstName,studentLastName,studentAddress,studentI
D)

    self.__SupervisorFirstName= sFirstName
    self.__SupervisorLastName= sLastName

#Setter
def setSupervisorFirstName(self,sFirstName):
    self.__SupervisorFirstName = sFirstName

def setSupervisorLastName(self, slastName):
    self.__SupervisorLastName= slastName
#Getter
def getSupervisorFirstName(self):
    return self.__SupervisorFirstName

def getSupervisorLastName(self):
    return self.__SupervisorLastName

def __str__(self):
    #Using base(grandparent) method
    return (f"\nGraduated Student Name is {self.getFirstName()}
{self.getLastName()}\n"
           f"Address is {self.getAddress()}\n"
           f"Student ID: {self.getStudentID()}\n"
           #sub class method with current object
           f"Supervisor's name: {self.getSupervisorFirstName()}
{self.getSupervisorLastName()}")

p = Person("Abdur","Rakib","UWE Frenchay Campus 4QXX")
print(p.__str__())
#Default lecturer name, address with id: 1
lec = Lecturer(1)
print(lec.__str__())
#Argumented passed to the constructor
lec2= Lecturer(2,"Chris","Simons","UWE Frenchay Campus 4QXX")
print(lec2.__str__())

```

```
student= Student("Peter","Miller","UWE Frechay Campus",5678)
print(student.__str__())

gradStudent= GraduateStudentClass("Dan","Fielding","UWE Frenchay
Campus",6789,"Jim","Smith")
print(gradStudent.__str__())
```

Output:

```
PS C:\TBC\Sem2\POP_Project> python -u C:\TBC\Sem2\POP_
sk2_2.py"

Person's name is Abdur Rakib
Address is UWE Frenchay Campus 4QXX

Lecturer Name is Mr/Ms X
Address is Bristol
Lecturer ID: 1

Lecturer Name is Chris Simons
Address is UWE Frenchay Campus 4QXX
Lecturer ID: 2

Student Name is Peter Miller
Address is UWE Frechay Campus
Student ID: 5678

Graduated Student Name is Dan Fielding
Address is UWE Frenchay Campus
Student ID: 6789
Supervisor's name: Jim Smith
PS C:\TBC\Sem2\POP_Project>
```

